

Enumerated Types and Strings

Where We Left Off

Integers, structs, and arrays hold data

Functions, loops, and I/O process it

Two gaps remain:

- a value that is one of a **fixed set of choices**

- a value that is **text**

Two New Ideas

Enumerated types - enums

a type whose values are a list of **names**

Strings

a sequence of **characters**: text as data

At the end we combine them:

turning text commands into enums

Magic Numbers

```
int state = 2 ;
```

What does 2 mean?

0	off	3	low pressure
1	warming up	4	sensor fault
2	ok		

Only the programmer knows

#define Helps a Little

```
#define STATE_OFF0  
#define STATE_OK 2
```

```
state = STATE_OK ; readable  
state = 42 ;           still compiles
```

The compiler sees plain integers

In a debugger, a print will show 2, not
STATE_OK

An Enumerated Type

A **new type** whose values are a fixed list of **names**

The compiler knows the whole list

The debugger shows the **name**

Same idea in C, C++, and Rust

enum in C

```
typedef enum {  
    STATE_OFF,  
    STATE_WARMING,  
    STATE_OK,  
    STATE_LOW,  
    STATE_FAULT  
} gauge_state_t ;  
  
gauge_state_t state = STATE_OFF ;
```

C Enums Are Integers

The names get the values 0, 1, 2, 3, 4

An enum variable is stored as an **int**

state = 42 ; **still compiles**

The names are **global** in their scope - hence the
STATE_ prefix

Choosing the Values

```
typedef enum {  
    MODE_IDLE      = 0x0,  
    MODE_SAMPLE    = 0x1,  
    MODE_STREAM     = 0x5  
} adc_mode_t ;
```

Match the codes the **hardware** expects

Enums as Bit Flags

```
typedef enum {  
    FLAG_VALID      = 1u << 0,  
    FLAG_ALARM      = 1u << 1,  
    FLAG_LOGGED     = 1u << 2  
} sample_flags_t ;
```

```
flags = FLAG_VALID | FLAG_ALARM ;  
if ( flags & FLAG_ALARM ) { ... }
```

switch on an Enum

```
switch ( state ) {  
    case STATE_OK : led_green ( ) ; break ;  
    case STATE_LOW :          led_red ( ) ;  
break ;  
    case STATE_FAULT :          led_blink ( ) ;  
break ;  
    default :          led_off ( ) ; break ;  
}
```

Jump straight to the matching **case**

switch Pitfalls

Forget **break** and execution **falls through** into the next case

Leave out a **default**, and -Wall warns about any enum value you did not handle

Keep that warning: it catches the case you forgot when a new state is added

Enums in C++: enum class

```
enum class GaugeState {  
    Off, Warming, Ok, Low, Fault  
};
```

```
GaugeState s = GaugeState::Ok ;
```

Names are **scoped**: GaugeState::Ok

No prefixes needed

enum class Is Strongly Typed

```
s = 2 ;                error  
int n = s ;            error  
int n = static_cast<int> ( s ) ; allowed
```

Choose the storage size:

```
enum class Mode : uint8_t { Idle, Sample } ;
```

Prefer enum class in new C++ code

Enums in Rust

```
enum GaugeState {  
    Off, Warming, Ok, Low, Fault,  
}
```

```
let s = GaugeState::Ok ;
```

Scoped names, like C++

Not an integer - convert with **as** if needed

match on an Enum

```
match s {  
  GaugeState::Ok      => led_green ( ),  
  GaugeState::Low     => led_red  ( ),  
  GaugeState::Fault   => led_blink ( ),  
  _                   => led_off  ( ),  
}
```

_ matches anything not listed

match Must Cover Every Case

Remove the `_` line and forget `Fault` :

```
error: non-exhaustive patterns:  
GaugeState::Fault not covered
```

Add a new state later, and **every** match that does not handle it stops compiling

Rust Enums Can Carry Data

```
enum Reading {  
    Valid ( f32 ),  
    OutOfRange { counts : u32 },  
    SensorFault,  
}
```

Each variant can hold **different** data
A struct and a tag in one type

Matching Variants With Data

```
match r {  
  Reading::Valid ( psi ) =>  
    println! ( "{:.1} psi", psi ),  
  Reading::OutOfRange { counts } =>  
    eprintln! ( "bad counts {}", counts ),  
  Reading::SensorFault =>  
    eprintln! ( "sensor fault" ),  
}
```

You Have Already Used These

`enum Option<T> { Some ( T ), None }`
a value that may be missing

`enum Result<T, E> { Ok ( T ), Err ( E ) }`
success or failure

`parse ( ), find ( ), read_line ( )` all return them

The C Version: Tagged Union

```
typedef struct {  
    reading_kind_t kind ;           the tag  
    union {  
        float psi ;  
        uint32_t counts ;  
    } data ;  
} reading_t ;
```

Nothing stops you reading `psi` when `kind` says
`counts`

Enums in Memory

- C: an int - usually 4 bytes
- C++: the underlying type - int unless you choose another, like `uint8_t`
- Rust: the compiler picks a small size - usually **1 byte** for a plain enum
with data: the tag + the largest variant

Enums in the Debugger

```
(gdb) print state  
$1 = STATE_OK
```

Not `$1 = 2`

The debug information records every name and its value

Example: Classify a Reading in C

```
gauge_state_t classify ( float psi ) {  
    if ( psi < 0.0f || psi > 150.0f )  
        return STATE_FAULT ;  
    if ( psi < 10.0f )  
        return STATE_LOW ;  
    return STATE_OK ;  
}
```

Returns a **name**, not a number

Example: Classify a Reading in Rust

```
fn classify ( psi : f32 ) -> GaugeState {  
    if psi < 0.0 || psi > 150.0 {  
        GaugeState::Fault  
    } else if psi < 10.0 {  
        GaugeState::Low  
    } else {  
        GaugeState::Ok  
    }  
}
```

Enums Compared

	C	C++ enum class	Rust
Names scoped	no - use a prefix	yes	yes
Converts to int	silently	only with a cast	only with <code>as</code>
Any int accepted	yes	no	no
Missing case	warning	warning	error
Carries data	no	no	yes

Strings: Text as Data

A **string** is a sequence of characters

Everything a user types is a string

Every line of a file is a string

Every command over a serial port is a string

Programs spend a lot of time on text

Characters Are Numbers

A character is stored as a small integer

The **ASCII** code gives each one a number

'A' is 65 '0' is 48

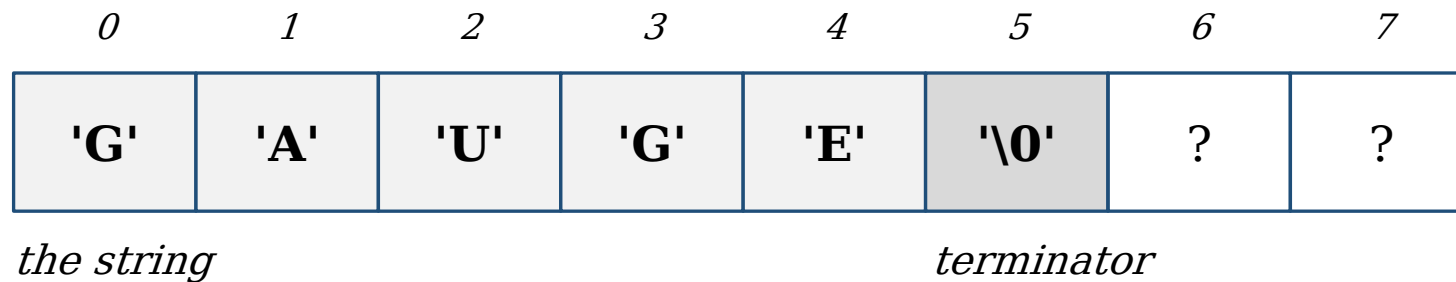
'0' + 5 is '5' 'A' + 32 is 'a'

Some ASCII Codes

Character	Decimal	Hex
'A'	65	0x41
'a'	97	0x61
'0'	48	0x30
' ' space	32	0x20
'\n' newline	10	0x0A
'\0' null	0	0x00

A String in C

An **array of char**, ending with a zero byte



```
char name[8] = "GAUGE" ;
```

```
strlen ( name ) = 5      sizeof ( name ) = 8
```

The Null Terminator

'\0' - the byte with value 0 - marks the end

C strings do **not** store their length

Every function walks until it finds the 0

"GAUGE" needs **6** bytes, not 5

Forget the terminator and functions read on into whatever memory comes next

String Literals

`const char *p = "GAUGE" ;`
points to **read-only** memory in the program

`char buf[] = "GAUGE" ;`
a **writable** copy in an array

Writing through `p` is undefined -
usually a crash

The C String Library

```
#include <string.h>
```

strlen	length, not counting the \0
strcmp	compare two strings
strcpy	copy - no size check
strcat	append - no size check
strchr strstr	find a character or substring

Length vs Size

```
char buf[64] = "psi" ;
```

<code>strlen (buf)</code>	3	characters before <code>\0</code>
<code>sizeof (buf)</code>	64	bytes in the array

strlen walks the string every time it is called
Do not call it on every pass of a loop

Comparing Strings in C

`if ( name == "GAUGE" )` compares **addresses**

`if ( strcmp ( name, "GAUGE" ) == 0 )` compares
contents

`strcmp` returns 0 when equal, negative or positive for
alphabetical order

Copying Strings in C

`buf = "GAUGE" ;` **error:** arrays cannot be assigned

`strcpy ( buf, src ) ;` no size check

Safer:

`snprintf ( buf, sizeof buf, "%s", src ) ;`

`strncpy` may leave `buf` **without** a terminator

Buffer Overflow

```
char name[8] ;  
strcpy ( name, "OIL_PRESSURE_GAUGE" ) ;
```

19 bytes into an 8-byte array

11 bytes overwrite **whatever is next**

The most famous security bug in C
gets () was removed from C entirely for this

Joining Strings in C

`strcat ( msg, label ) ;` same danger as `strcpy`

Build the whole message at once:

```
char msg[64] ;  
snprintf ( msg, sizeof msg,  
           "%s: %.1f psi", label, psi ) ;
```

Numbers and Strings in C

Text to number:

```
long n = strtol ( s, &end, 10 ) ;  
double d = strtod ( s, &end ) ;
```

Number to text:

```
snprintf ( buf, sizeof buf, "%.1f", psi ) ;
```

atoi () exists but cannot report bad input

Walking a String in C

```
#include <ctype.h>
```

```
int digits = 0 ;  
for ( int i = 0 ; s[i] != '\0' ; i++ ) {  
    if ( isdigit ( (unsigned char) s[i] ) )  
        digits++ ;  
}
```

isdigit isalpha isspace toupper tolower

Splitting a Line in C

```
char line[] = "1456,50.0,12345" ;  
char *field = strtok ( line, "," ) ;  
while ( field != NULL ) {  
    /* use field */  
    field = strtok ( NULL, "," ) ;  
}
```

strtok **writes** \0 over each comma -
the line must be a writable array

std::string in C++

```
#include <string>
```

```
std::string name = "GAUGE" ;
```

Knows its own **length**

Grows as needed

Frees its memory at the end of its **scope**

std::string Operations

<code>name.size ()</code>	5
<code>name + "-1"</code>	"GAUGE-1"
<code>name == "GAUGE"</code>	compares contents
<code>name[0]</code>	'G'
<code>name.substr (0, 3)</code>	"GAU"
<code>name.find ("UG")</code>	2

`find` returns `std::string::npos` when not found

Building Strings in C++

```
std::string msg = label ;  
msg += ": " ;  
msg += std::to_string ( psi ) ;
```

C++20:

```
auto msg = std::format ( "{ }: {:.1f} psi", label,  
psi ) ;
```

No buffer size to get wrong

Numbers and Strings in C++

Text to number:

```
int n = std::stoi ( s ) ;  
float f = std::stof ( s ) ;  
throw an exception on bad input
```

Number to text:

```
std::to_string ( n )  
std::format ( "{:.1f}", psi )
```

Splitting a Line in C++

```
#include <sstream>
```

```
std::istringstream in ( line ) ;
```

```
std::string field ;
```

```
while ( std::getline ( in, field, ',' ) ) {
```

```
    // use field
```

```
}
```

getline with a delimiter stops at each comma

std::string_view

A **view** of someone else's characters:
a pointer and a length, no copy

```
void show ( std::string_view label ) ;
```

Accepts literals and std::string alike

Must not outlive the string it views

Passing Strings in C++

Read only, no copy:

```
void show ( const std::string &s ) ;
```

```
void show ( std::string_view s ) ;
```

Calling a C function:

```
puts ( s.c_str ( ) ) ;
```

`c_str ( )` gives a null-terminated `const char *`

Strings in Rust: Two Types

String

owned, growable, on the heap
like `std::string`

&str

a borrowed **view** of characters
like `std::string_view` - but checked

String Literals in Rust

```
let s : &str = "GAUGE" ;
```

Stored in the program itself
Lives for the whole program

Read only - enforced by the **type**
not by a crash

Making a String

```
let s = String::from ( "GAUGE" ) ;  
let s = "GAUGE".to_string ( ) ;
```

```
let mut s = String::new ( ) ;  
s.push_str ( "GAU" ) ;  
s.push ( 'G' ) ;  
s.push ( 'E' ) ;
```

String Operations in Rust

<code>s.len ()</code>	5 - in bytes
<code>s == "GAUGE"</code>	compares contents
<code>s.contains ("UG")</code>	true
<code>s.find ("UG")</code>	Some (2)
<code>s.starts_with ("GA")</code>	true
<code>s.to_lowercase ()</code>	"gauge"
<code>s.trim ()</code>	spaces removed at both
<code>ends</code>	

Joining Strings in Rust

```
let msg = format! ( "{ }: {:.1} psi", label, psi ) ;
```

```
let a = String::from ( "OIL" ) ;  
let b = a + "_PSI" ;
```

+ **moves** a into b - a can no longer be used

format! is usually clearer

Characters Are Not Bytes

Rust strings are **UTF-8**

"25°C".len () 5 - the degree sign takes 2
bytes

s[0] **error:** cannot index a string

for c in s.chars () each character
for b in s.bytes () each byte

Walking a String in Rust

```
let mut digits = 0 ;  
for c in s.chars ( ) {  
    if c.is_ascii_digit ( ) {  
        digits += 1 ;  
    }  
}
```

No terminator to find - the length is known

Splitting a Line in Rust

```
let line = "1456,50.0,12345" ;
```

```
for field in line.split ( ',' ) {  
    // field is a &str - a view into line  
}
```

```
let parts : Vec<&str> = line.split ( ',' ).collect ( ) ;
```

Numbers and Strings in Rust

Text to number:

```
let counts : u32 = parts[0].parse ( )? ;  
let psi = parts[1].parse::<f32> ( )? ;  
returns a Result
```

Number to text:

```
psi.to_string ( )  
format! ( "{:.1}", psi )
```

Passing Strings in Rust

Take a **&str** parameter:

```
fn show ( label : &str ) { ... }
```

```
show ( "GAUGE" ) ;    a literal  
show ( &name ) ;      a String, borrowed
```

One function serves both

Strings in Memory

C: `char name[] / char *`



C++: `std::string`



Rust: `String`



Rust: `&str`



a view into the String above

Strings in the Debugger

```
(gdb) print name  
$1 = "GAUGE"
```

```
(gdb) print name[0]  
$2 = 71 'G'
```

```
(gdb) x/8cb name  
71 'G' 65 'A' 85 'U' 71 'G' 69 'E' 0 '\000' ...
```

rust-gdb shows a String as its text

Enums and Strings Together

Commands arrive as **text** over a serial port:

"START" "STOP" "STATUS"

Convert text to an **enum** once, where it arrives

The rest of the program uses the enum

Text to Enum in C

```
typedef enum { CMD_START, CMD_STOP,  
               CMD_STATUS, CMD_UNKNOWN } command_t ;  
  
command_t parse_command ( const char *s ) {  
    if ( strcmp ( s, "START" ) == 0 )    return  
    CMD_START ;  
    if ( strcmp ( s, "STOP" ) == 0 )     return  
    CMD_STOP ;  
    if ( strcmp ( s, "STATUS" ) == 0 )   return  
    CMD_STATUS ;  
    return CMD_UNKNOWN ;  
}
```

Enum to Text in C

```
static const char *STATE_NAMES[] = {  
    "OFF", "WARMING", "OK", "LOW", "FAULT"  
} ;
```

```
printf ( "%s\n", STATE_NAMES[state] ) ;
```

The enum values 0 to 4 are the **index**
Keep the list in the **same order** as the enum

Text to Enum in C++

```
enum class Command { Start, Stop, Status, Unknown }  
;
```

```
Command parse_command ( std::string_view s ) {  
    if ( s == "START" ) return Command::Start ;  
    if ( s == "STOP" )  return Command::Stop ;  
    if ( s == "STATUS" ) return Command::Status ;  
    return Command::Unknown ;  
}
```

Text to Enum in Rust

```
fn parse_command ( s : &str ) -> Option<Command>
{
    match s.trim ( ) {
        "START"      => Some ( Command::Start ),
        "STOP"       => Some ( Command::Stop ),
        "STATUS"     => Some ( Command::Status ),
        _            => None,
    }
}
```

Enum to Text in Rust

```
#[derive(Debug)]  
enum GaugeState { Off, Warming, Ok, Low, Fault }  
  
println! ( "{:?}", s );           Ok
```

Or a match that returns a &str :

```
GaugeState::Ok => "OK",
```

Why Convert at the Edge?

Strings inside a program are:

slow to compare, easy to misspell,
unchecked

Enums are:

one integer compare, checked by the
compiler,
and every case must be handled

Parse text **once** where it enters the program

Strings Compared

	C	C++	Rust
Type	char []	std::string	String, &str
Knows its length	no - finds \0	yes	yes
Grows	no	yes	yes (String)
Compare	strcmp	==	==
Join	snprintf	+ std::format	format!
Split	strtok	getline (in, f, ',')	split (',')
To a number	strtol	std::stoi	parse ()

Good Habits

Use an **enum**, not a magic number

C: prefix enum names. C++: use enum class

Never copy text into a fixed buffer without its **size**

Take **&str** or **string_view** parameters

Convert text to an enum at the edge

What We Can Now Do

Name a fixed set of choices with an enum

switch or **match** on every case

Store, compare, split, and convert text

Turn text commands into enums

Let's practice

Exercise: Using **stringpractice**